

Chapter 11

Tools for Scientific Computing

The purpose of this book is to prepare you to work in data science, the natural sciences and engineering. We have addressed programming concepts and the Python language. In this chapter we will examine some of the tools used in scientific and engineering computations.

11.1 Numpy

Assume that you have collected temperature data for a project and that you have 10,000 collected data points. It would be absurd to write out 10,000 different variables to hold the values: [a,b,c,d,e,f, ..., z, aa, ab, ac, ...]. In this case using just lower case, you would need to go up to three letter variables (aaa, aab, aac, ...). There is not a concern about running out of variable names. The concern is that there is not a good way to access all of those variable names. It cannot be easily accessed in a loop.

What we want is something like the vector idea found in math courses where you have $x_1, x_2, x_3, \dots, x_{10000}$. Most programming languages support this through a data structure called an array. These languages extend the basic datatype to an array version. Meaning you can have arrays of integers, floats and other more exotic data containers. These arrays must have a uniform data type for the entire array. Python does not include a basic array structure. It does have a more flexible structure known as a list which allows for elements of different data types. Lists were covered in an earlier chapter.

To get a traditional array in Python, we use the NumPy module which is a collection of objects used to create and manipulate numerical data. NumPy provides an extension package to Python for multi-dimensional arrays. Numpy is written in C and often optimised for special hardware and so core Numpy routines are at least an order of magnitude faster. The design of the package reflects an array oriented computing view and is designed for scientific computing. Numpy is very fast and very efficient. Although Python can be rather slow, Numpy can achieve speeds associated with compiled languages like C.

Information and download of Numpy are at: <https://numpy.org/>. Numpy is part of the Anaconda distribution.

We begin with creating a simple array of 32 bit integers.

```
import numpy as np
a = np.array([0, 1, 2, 3])
print(a)
```

```
[0 1 2 3]
```

Note that when printing the array it looks like a list, but without the commas separating the values.

We can get information about the datatype through

```
print(type(a))
print(a.dtype)
```

```
<class 'numpy.ndarray'>
dtype('int32')
```

An example of a two dimensional array

```
b = np.array([[0, 1, 2], [3, 4, 5]])
print(b)
```

```
[[0 1 2]
 [3 4 5]]
```

```
b.shape
```

```
(2, 3)
```

Numpy arrays are zero indexed. This means that the index for the array starts with 0 and not 1. Access is via square brackets as show here

```
d = np.array([47, 19, 23, 80])
print(d[2])
```

```
23
```

We typically work with integer or floating point arrays. If all the elements are integers then that will be the datatype for the whole array, otherwise we get an array of floats:

```
d = np.array([47, 19, 23, 80])
print(d.dtype)
e = np.array([47, 19, 23, 80.0])
print(e.dtype)
```

```
int32
float64
```

In practice we don't hand enter the data. It is either read in from a file (see chapter 10) or it is created by an array generation function. Some very useful functions to create Numpy arrays follow.

- To create a range of numbers starting at a , up to but not including b with a separation of step,
`np.arange(a,b,step)`
- To create a range of N numbers starting at a , ending at b ,
`np.linspace(a,b, N)`
- To create an array of zeros with shape (n,m) ,
`np.zeros((n,m))`

- To create an array of ones with shape (n, m) ,
`np.ones((n,m))`
- To create an array of random numbers between 0 and 1 with shape (n, m) ,
`np.random.rand(n,m)`

Please experiment to get a feel for Numpy arrays.

```
print(np.linspace(0,5,50))
```

We saw above that one can access array elements using the square brackets. This can be used to assign values

```
x = np.zeros((5))
x[2] = 512
print(x)
```

```
[ 0.  0. 512.  0.  0.]
```

Numpy has many feature for array access and manipulation. Accessing parts of the array are known as slices. One such slice is the range operation: `array[start:end:step]` (begin at “start”, go up to but one less than “end” and use “step” as the stride):

```
a = np.arange(10)
print(a)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
print(a[2:5])
```

```
[2, 3, 4]
```

The slice is a view of the data - not a copy.

```
a = np.random.randint(0,51,15) # random array
print("a = ", a) # print this out
d = a[3:6] # grab a slice
print("d = ", d) # print out the slice
d[0] = 100 # set one of the elements in the slice
print("Modified d = ", d) # print d again to see the modification
print("a = ", a) # print a to see if it was changed ...
```

```
a = [17, 10, 2, 50, 12, 6, 34, 29, 43, 14, 44, 48, 28, 32, 40]
d = [50, 12, 6]
Modified d = [100, 12, 6]
a = [ 17, 10, 2, 100, 12, 6, 34, 29, 43, 14, 44, 48, 28, 32, 40]
```

To make a copy you should use the `copy()` method so you don’t modify the original array:

```
c = a[i:j].copy()
```

Arrays and loops go hand in hand. A for loop is an easy way to access or modify the elements of an array. For example, fill out an array of length 100 with $1/(i+1)$ where i is the index.

```
N = 100
x = np.zeros((N))
for i in range(N):
    x[i] = 1/(i+1)
```

11.2 Numpy operations

Python is known to be slow. This partially has to do with the dynamic variables and the challenges of optimization in this environment. So, Numpy has a number of element-wise operations built-in. The basic binary operators are overloaded which means that when Python sees $x + y$ it knows to call the element-wise addition function (under the hood). Automatic element-wise operations include

- Addition of arrays: $x + y$
- Addition with constants: $x + 1.0$
- Scalar multiplication: $c * x$
- Array multiplication: $a * b$
- Matrix multiplication (dot): `np.dot(x,y)`
- Functions $x * x + 2 * x + 3$, `np.sin(x)`, `np.exp(x)`

These operations are very fast. This is where the power of Numpy starts to emerge. If you want to create a collection of points (x_k, y_k) where $0 \leq x_k < 4\pi$, $y_k = x_k^2 + e^{-x_k} \sin(x_k)$, with $0 < k < 200$, the traditional element-wise approach would be

```
import math
N = 200
x = np.empty((N)) # declare arrays (default is float)
y = np.empty((N))
delta = 4*math.pi/N
for i in range(N):
    x[i] = i*delta
    y[i] = x[i]*x[i] + math.exp(-x[i])*math.sin(x[i])
```

Using the overloaded operators in Numpy, we can rewrite this using the implicit element-wise operations:

```
import numpy as np
N = 200
x = np.linspace(0,4*np.pi,N)
y = x*x + np.exp(-x)*np.sin(x)
```

Because these arrays are objects, they have methods associated with them.

```
x.sum()
x.mean() # Average
x.std() # Standard Deviation
x.max()
x.min()
```

Elementwise logical operations and comparisons can be done with Numpy arrays as well. We will see more Numpy in a later section on plotting.

11.3 Random Values

When doing simulations of natural systems, or testing code, or performing numerical optimization, it is important to have access to random numbers. Python and Numpy provide values sampled from a variety of distributions. The generation of random values is a whole separate subject and one needs to keep in mind that the values produced by these routines

are not truly random and are not appropriate for cryptographic level randomness. However, many applications work very well with pseudo-random sequences. Random and Numpy Random libraries both use the Mersenne twister sequence to generate their values and in most cases only one library is required.

The Random number library is accessed via `import random` and the Numpy random routines are brought in with `import numpy`. The Random library is accessed via `random.function()` and the Numpy random sample library is accessed via `numpy.random.function()`.

A few routines in Random:

- `randint(low, high)` - Produces a random integer n where $low \leq n \leq high$.
For example, `randint(0,5)` produces a number from the set $\{0, 1, 2, 3, 4, 5\}$.
- `choice(x)` - Select from list x .
- `shuffle(x)` - Shuffle the list x . Will modify x .
- `sample(x, k)` - Returns a shuffled subset from x but leaves x unmodified. Default $k = len(x)$
- `random()` - Return a floating point number from $[0, 1.0)$.
- `normalvariate(mu, sigma)` - Return a number from the Normal distribution where μ is the mean, and σ is the standard deviation.

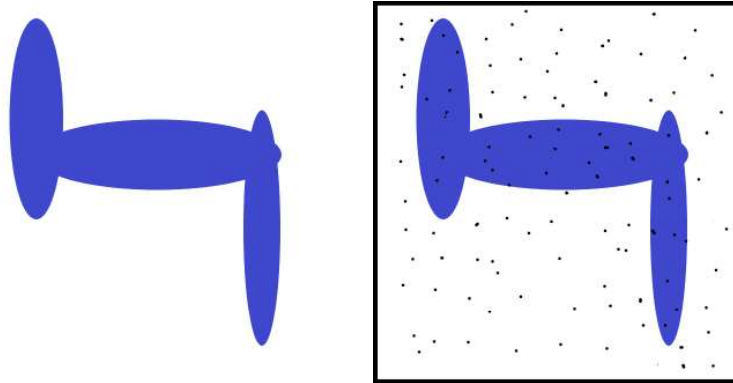
A few routines in Numpy.Random:

- `randint(low, high)` - Produces a random integer n where $low \leq n < high$.
For example, `randint(0,5)` produces a number from the set $\{0, 1, 2, 3, 4\}$.
- `random(size)` - Produces an array, shape given by size, of uniform random numbers drawn from $[0, 1)$.
`random((3,2))` produces a 3×2 matrix of random values.
- `normal(mu, std, shape)` - Produces samples from a normal distribution with a given mean, μ , and standard deviation, σ , with array shape given by the tuple shape.

11.3.1 Example: Computing Area

In calculus you learn some very powerful methods to compute many geometric quantities such as area. The methods require that you are able to compute the antiderivative of a function. This may or may not be possible based on the formulas that describe the region of interest. There are numerical (approximate) methods that can give you an estimate when the antiderivative eludes you. This program implements one such method. This particular approach does not need any background in calculus ... only inebriated dart players :)

Say you have some area in the plane (figure on the left), and you toss a bunch darts, random points, in some box that holds the region we care about (figure on the right):



[Note that parts of the blue region can fall outside the box - it will just not be included in the area computation.] After lots of darts, count the number of points that fall in the area, call this *Inside*. Count the total number of points and call that *Total*. Compute the area of the square, call it *SqArea*. The approximation of the area is:

$$\text{Area of figure} \approx \left(\frac{\text{Inside}}{\text{Total}} \right) * \text{SqArea}$$

For our code example, we will use two ellipses to build the figure. We will prompt the user for the information about the two ellipses, and then print out the area. In addition, we will restrict the area estimation to the square $0 < x < 10$ and $0 < y < 10$.

```
import random

def inEllipse(h,k,a,b,x,y):
    inside = (((x-h)*(x-h))/(a*a) + ((y-k)*(y-k))/(b*b)) < 1:
    return inside

def main():
    count = 0
    N = 100000
    h1, k1, a1, b1 = eval(input("Please enter h1, k1, a1, b1:"))
    h2, k2, a2, b2 = eval(input("Please enter h2, k2, a2, b2:"))
    for i in range(N):
        x = random.uniform(0,10)
        y = random.uniform(0,10)
        result1 = inEllipse(h1,k1,a1,b1,x,y)
        result2 = inEllipse(h2,k2,a2,b2,x,y)
        if result1 == True or result2 == True:
            count += 1
    Area = 100.0*(float(count)/float(N))
    print("The area of the region is approximately", Area)

main()
```

11.4 Excel

For many applications in Data Science, the data will arrive in an Excel document. There are two approaches to moving the data from Excel to Python and back: first is using an interchange format such as comma separated values and the second is reading and writing the Excel formatted files. The second approach uses external modules (xlrd) to bring in the file. We will focus on the first approach in this text.

The interchange format most commonly use is CSV - comma separated values. A spreadsheet is basically a matrix of cells which are containers for general data. A CSV file writes out each row separating row items by commas.

R1C1	R1C2	R1C3	R1C4
R2C1	R2C2	R2C3	R2C4
R3C1	R3C2	R3C3	R3C4

export as CSV $\Rightarrow$

R1C1,	R1C2,	R1C3,	R1C4 \n
R2C1,	R2C2,	R2C3,	R2C4 \n
R3C1,	R3C2,	R3C3,	R3C4 \n

Assume the CSV files looks like

```
0.54,6.4,5.5,2.3,3.4
1.2,4.6,54.7,17.9,4.12
44.4,2,4,-4.8,66
```

Recall to read a text file:

```
fin = open('data.csv')
for line in fin:
    row = line.strip()
    print(row)
fin.close()
```

```
0.54,6.4,5.5,2.3,3.4
1.2,4.6,54.7,17.9,4.12
44.4,2,4,-4.8,66
```

How would we get these values loaded into a Numpy two-dimensional array? Problem one is that a row is a single object, not a list of values. Second issue is that they come in as text.

```
fin = open('data.csv')
for line in fin:
    row = line.strip()
    items = line.split(",")
    print(items)
fin.close()
```

```
['0.54', '6.4', '5.5', '2.3', '3.4']
['1.2', '4.6', '54.7', '17.9', '4.12']
['44.4', '2', '4', '-4.8', '66']
```

Each line needs to be split and the data converted to floats. Then it can be loaded into a numpy array. If we know the size of the matrix, then it is fairly straightforward:

```
rows = 0
data = np.empty((3,5),dtype="float")
fin = open('data.csv')
for line in fin:
    row = line.strip()
    items = row.split(",")
    cols = 0
    for item in items:
        data[rows,cols] = eval(item)
        cols = cols + 1
    rows = rows + 1
print(data)
fin.close()
```

If you don't know the dimensions then a small modification will work:

```

rows = 0
data = np.array([], dtype="float")
fin = open('data.csv')
for line in fin:
    row = line.strip()
    items = row.split(",")
    rows = rows + 1
    cols = 0
    for item in items:
        data = np.append(data, [eval(item)])
        cols = cols + 1
matrix = data.reshape([rows, cols])
print(matrix)
fin.close()

```

Or you can just do

```

from numpy import genfromtxt
data = genfromtxt('data.csv', delimiter=',')

```

or

```

from pandas import read_csv
df = read_csv('data.csv')

```

11.5 Matplotlib

There are a number of plotting options for Python. One of the oldest plotting packages and part of the SciPy collection is Matplotlib. You can think of this as an array visualization tool and not just a tool to produce graphs. This section will focus on traditional 2D plots with the goal of publication quality plots. Other applications such as 3D plots, animated plots, etc will not be addressed here. We have included references to the Matplotlib documentation and references at the end.

In order to use the routines you need to import the plotting package:

```
import matplotlib.pyplot as plt
```

For our first example, we will plot a simple data set:

```

x = np.array([0,1,2,3,4])
y = np.array([1,4,3,7,9])
plt.plot(x,y)
plt.show()

```

The resulting plot is shown in Figure 11.1(a). It is that simple. The general idea is that you create the x and y arrays from your code (data analysis, simulation, etc). These are passed into the plot command combined with any decorations you want. Matplotlib then renders the graphics. When using Jupyter Lab it attempts to render any of the plot commands, even a title command, plotting works differently than in the interpreter. So, you will normally place all of your plot commands related to a given plot in a single cell.

Another example is given in Figure 11.1(b).

```

x = np.linspace(0,1,100)
y = x*(1-x) # Inverted parabola
plt.plot(x,y, color = 'green', linestyle = 'dotted')
plt.show()

```